



UNIVERSIDADE FEDERAL DO PARANÁ
SETOR DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANTONIO AMORIM DO NASCIMENTO NETO (GRR20255289)
DOUGLAS EDUARDO PUCHALSKI DOS SANTOS (GRR20250662)
EDUARDO FERRUZZI PRESSUTO (GRR20252190)
LUCAS WESLEY FERRAZ PEPES (GRR20255692)

ATIVIDADE 8 – Projeto DW

Curitiba/PR

2026

1 - Introdução

1.1 - Contexto:

A empresa analisada atua no setor varejista e possui várias lojas físicas distribuídas por diferentes cidades e estados, conforme pode ser observado no modelo relacional do banco de dados BD_VAREJO.

O banco de dados possui informações relacionadas as lojas, seus endereços, cidades e estados, além de funcionários, seus respectivos cargos e históricos de promoções e comissões. Também, são armazenados os dados dos clientes, tanto ativos quanto antigos, junto com seus logins de acesso.

Em relação aos produtos, o sistema possui um catálogo dividido em três categorias principais: alimentos, eletrônicos e vestuários. A empresa também mantém informações sobre seus fornecedores e sobre as operações realizadas no dia a dia. Entre essas operações, estão as vendas e compras, incluindo seus respectivos itens, o que permite controlar a entrada e saída de mercadorias e acompanhar o faturamento por loja, funcionário e cliente.

Por se tratar de um banco de dados voltado para as operações do dia a dia, sua estrutura é bem normalizada e focada no processamento das transações. Dessa forma, o BD_VAREJO funciona como a principal fonte de dados para a construção de uma solução analítica, que poderá ser usada para gerar informações e auxiliar na tomada de decisões.

1.2 - Problemas e abordagens existentes:

Banco de dados do tipo operacionais, como o BD_VAREJO, são desenvolvidos para garantir integridade, consistência e desempenho em operações frequentes de inserção, atualização e exclusão. Essas características são essenciais para o funcionamento diário na empresa, porém, podem se tornar limitantes quando o objetivo passa a ser a realização de análises gerenciais e as tomadas de decisões estratégicas.

Consultas analíticas que envolvem vendas, compras, produtos, funcionários, cliente e lojas ao longo do tempo precisam realizar múltiplas junções entre várias tabelas normalizadas. Isso pode acarretar num processamento mais custoso e pouco flexível para a geração de relatórios, ou para a utilização de outras ferramentas de negócio.

O cenário torna-se mais complexo pelo fator de que a origem dos dados nem sempre apresenta confiabilidade ou integridade. Por exemplo, o arquivo de backup da produção BD_VAREJO.sql, que apresenta divergências em relação ao modelo entidade-relacionamento documentado em BD_VAREJO.png. Entre elas, a ausência das tabelas de itens de venda e de compra (tb018_itens_vendas e tb019_itens_compras) e as alterações na estrutura de tb010_012_vendas e tb012_017_compras (Seção 2).

Esse tipo de inconsistência entre a fonte de dados e o modelo utilizado como referência é comum em ambientes do mundo do trabalho e reforça a importância de um processo estruturado de extração, transformação e carga (ETL). O processo é necessário para tratar as divergências, garantir a qualidade dos dados e consolidar as informações antes de serem disponibilizadas para qualquer análise.

1.3 - Proposta:

Diante desse cenário, propõe-se o desenvolvimento de um Data Warehouse para a rede varejista, como o objetivo de reorganizar os dados provenientes de BD_VAREJO em um modelo dimensional. A estrutura será voltada para a análise de várias dimensões de vendas, compras e do desempenho operacional.

Isso inclui a definição dos fatos de interesse do negócio, a tabela fato_venda, juntamente com suas dimensões associadas. Entre elas, Tempo, Loja, Cliente, Funcionário, Tipo, Categoria etc. Essa organização permitirá a realização de consultas analíticas de maneira mais rápida e intuitiva, como a análise do volume de vendas por loja e período, o desempenho dos funcionários, comportamento de compra dos clientes etc.

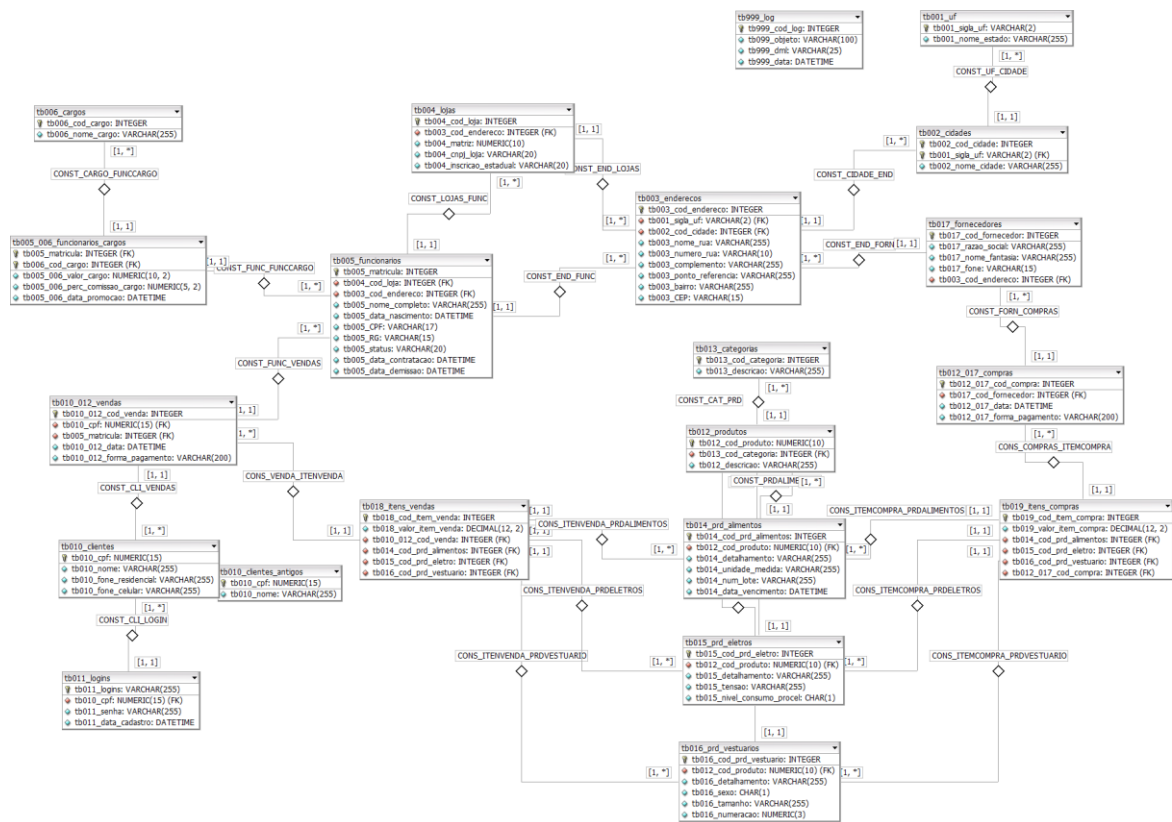
Para que essa estrutura seja viável, será necessário um processo de ETL responsável por tratar as inconsistências identificadas na fonte de dados, realizar a limpeza, padronização e integração das informações e, posteriormente, alimentar o modelo dimensional de forma confiável. Dessa maneira, o data Warehouse pode servir como base para a geração de relatórios gerenciais e outras ferramentas de negócio.

Este texto está organizado da seguinte forma: a Seção 2 apresenta o modelo relacional atual do BD_VAREJO e as divergências identificadas em relação ao backup real; a Seção 3 descreve o modelo conceitual e lógico do Data Warehouse proposto;

as seções seguintes tratarão da arquitetura de ETL, frequência de carga, hardware e SGBD recomendados, além da análise gerencial e conclusão.

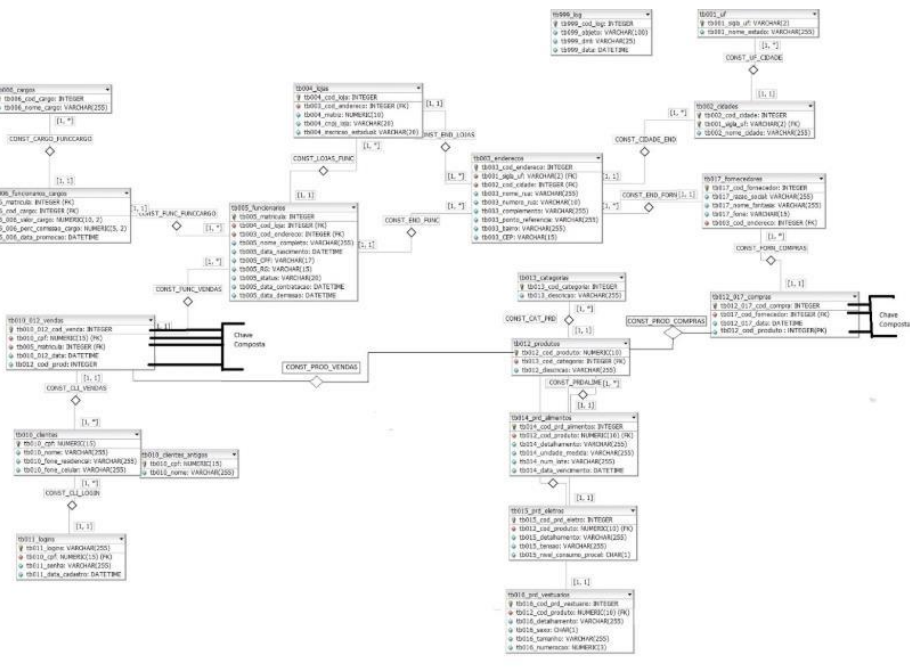
2 - Situação e Modelo Atual

FIGURA 1 – BD_VAREJO.png original



FONTE: Material Fornecido

FIGURA 2 – BD_VAREJO.png atualizada



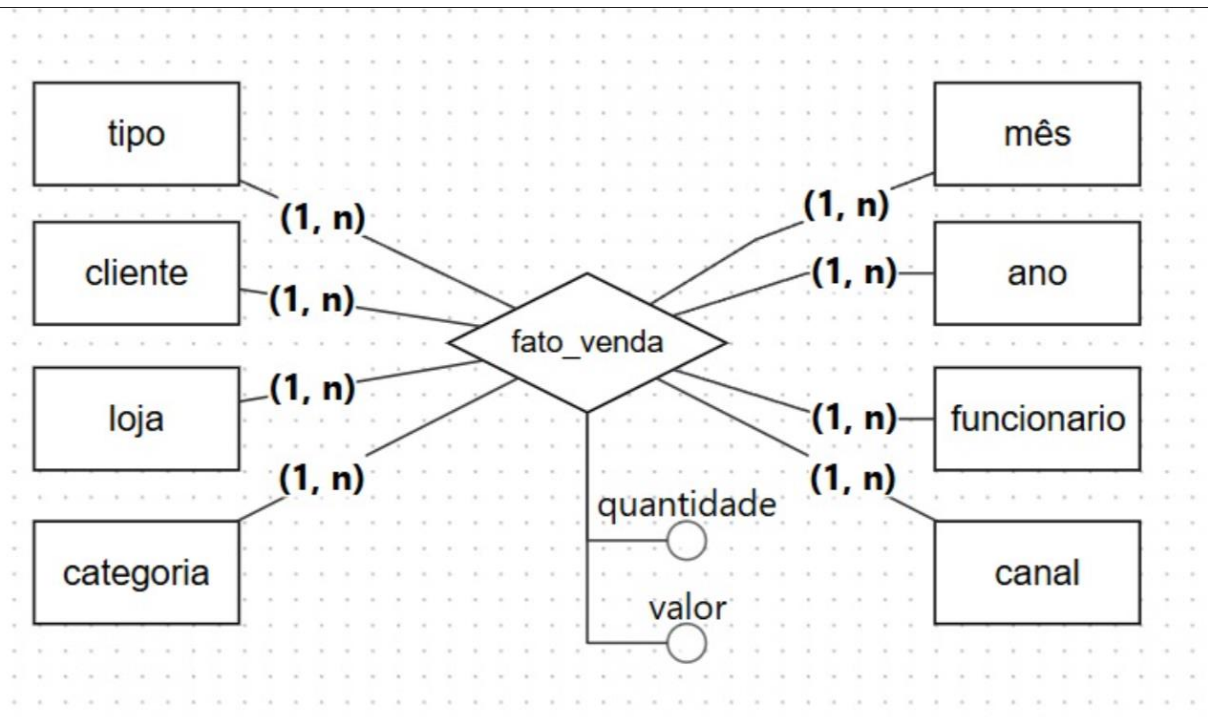
FONTE: Autores (2026)

Seguindo o arquivo BD_VAREJO.sql (backup do Banco de Dados já implementado), algumas alterações foram feitas na FIGURA 1, o resultado das alterações é a FIGURA 2. Mudanças realizadas:

- As tabelas tb018_itens_vendas e tb019_itens_compras foram excluídas;
- Na tabela tb010_012_vendas a coluna tb010_012_forma_pagamento foi removida, a coluna tb012_cod_produto foi adicionada e foi alterada de uma chave primária simples (somente coluna tb010_012_cod_venda) para uma chave primária composta (colunas: tb010_012_cod_venda, tb005_matricula, tb010_cpf e tb012_cod_produto);
- Na tabela tb010_012_vendas passaram a constar também as colunas de quantidade (tb010_012_quantidade) e valor unitário (tb010_012_valor_unitario), de modo que cada linha de venda representa um item vendido e o valor da venda é calculado por quantidade × valor unitário;
- Na tabela tb012_017_compras a coluna tb012_017_forma_pagamento foi removida, a coluna tb012_cod_produto foi adicionada e foi alterada de uma chave primária simples (somente coluna tb012_017_cod_compra) para uma chave primária composta (colunas: tb012_017_cod_compra, tb012_cod_produto, tb017_cod_fornecedor);
- Foi criado o relacionamento entre a coluna tb012_cod_produto na tabela tb010_012_vendas e a coluna tb012_cod_produto na tabela tb012_produtos;
- Foi criado o relacionamento entre a coluna tb012_cod_produto na tabela tb012_017_compras e a coluna tb012_cod_produto na tabela tb012_produtos;

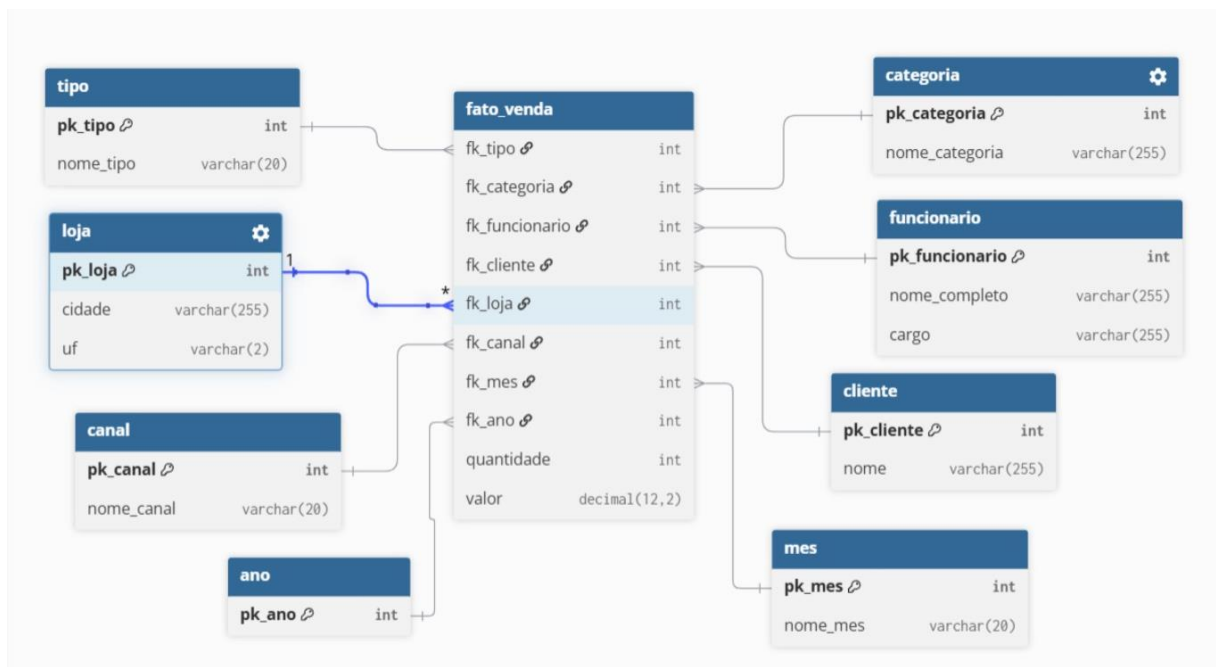
3. Modelo Conceitual e Lógico do BDM

Figura 3 – Modelo Conceitual do BDM



Fonte: Autores (2026)

Figura 4 – Modelo Lógico do BDM



Fonte: Autores (2026)

O Modelo Conceitual do BDM (FIGURA 3) foi montado em volta da tabela **fato_venda**, que registra os eventos de venda dos produtos. Essa tabela possui duas

medidas principais: quantidade e valor, para resumir o volume e o faturamento das vendas, cruzando essas informações com oito dimensões: tipo e categoria (que formam a hierarquia de produto), cliente, funcionário, loja, canal (separando o que é loja física do e-commerce) e mês/ano (hierarquia de tempo).

O BDM proposto trabalha com pré-agregação: cada linha da tabela fato já salva a quantidade e o valor somados para uma combinação de dimensões, assim como o que foi orientado nas aulas. Se a chave estrangeira de uma dimensão estiver com o valor NULL, significa que o valor considera todas as opções daquela dimensão. Essa estrutura foi pensada para responder às consultas gerenciais direto com um filtro WHERE, sem precisar fazer agregações (GROUP BY) no momento da busca, o que deixa o processamento bem mais leve.

No Modelo Lógico (FIGURA 4), essa estrutura foi transformada em tabelas relacionais: a tabela fato_venda junta as chaves estrangeiras e as medidas (todas opcionais), enquanto cada dimensão vira uma tabela simples com seus atributos descritivos. A dimensão produto, prevista na primeira versão do modelo, foi retirada por não ser utilizada por nenhuma das consultas gerenciais.

4 - Arquitetura de ETL

4.1 - Abordagem e ferramentas

O processo de ETL extrai os dados do BD_VAREJO, transforma eles para o modelo dimensional e após isso carrega no BDM. Foram consideradas diversas abordagens: carga a partir do arquivo de backup, carga direta do SQL Server para o BDM e carga passando por uma Staging Area/ODS. A carga a partir do arquivo SQL foi descartada, pois o BD_VAREJO já está implementado no SQL Server e o backup apresenta divergências em relação ao modelo documentado (Seção 2). Foi escolhida a abordagem pela carga direta do SQL Server para o BDM, sem Staging Area intermediária. Há uma única fonte de dados, e as transformações necessárias (agregação, definição do tipo de produto e do canal de venda) podem ser feitas na própria consulta de extração.

Para a implementação, foi desenvolvido o script dw.py, em Python, com as bibliotecas pyodbc (conexão ODBC com o SQL Server) e pymysql (conexão com o MySQL, banco dw_vendas, onde foram criadas as tabelas do DDL). Entre as ferramentas apresentadas (Sqllines, Fullconvert, ODBC, dblink, Pentaho, DBeaver etc.), a escolha por um script próprio se deve à necessidade de integrar dois SGBDs diferentes e de ter controle total sobre as regras de transformação, sem custo de licenciamento.

4.2 - Extração

A origem é uma instância local do SQL Server Express, banco ADS, acessada via ODBC Driver 17 com autenticação do Windows. A extração é feita por consultas SELECT específicas para cada dimensão e para cada agrupamento da tabela fato, trazendo apenas as colunas de interesse, e não uma réplica completa do banco.

4.3 - Transformação e carga das dimensões

Cada dimensão é carregada por uma função própria, que executa três passos: 1 - consulta os dados na origem; 2 - insere os registros na dimensão do MySQL, que gera a chave substituta; 3 - devolve um dicionário que associa a chave original (código, matrícula ou CPF) à nova chave do BDM. Esses dicionários são reaproveitados na carga da tabela fato para traduzir as chaves. As dimensões carregadas são:

- **tipo:** não existe tabela de origem, então os valores Alimento, Eletro e Vestuario são inseridos manualmente, correspondendo às três tabelas de produtos (tb014, tb015 e tb016).
- **categoria:** carregada a partir da tb013_categorias.
- **funcionário:** carregado a partir da tb005_funcionarios, com o cargo obtido por LEFT JOIN nas tabelas tb005_006_funcionarios_cargos e tb006_cargos, para que funcionários sem cargo não sejam perdidos. Como alguns funcionários possuem mais de um cargo (histórico de promoções), a dimensão recebe mais de uma linha para eles (67 linhas para 40 funcionários); a tabela fato utiliza apenas uma delas por matrícula.
- **cliente:** carregado a partir da tb010_clientes, usando o CPF como chave original.
- **loja:** carregada a partir da tb004_lojas, com junção nas tabelas de endereços e cidades para obter cidade e UF.
- **canal:** valores Loja Física e Loja Virtual são inseridos manualmente. A classificação de cada venda é feita na extração: se o CPF do cliente possui login em tb011_logins, a venda é Loja Virtual; caso contrário, Loja Física. Como todos os clientes possuem login, todas as vendas foram classificadas como Loja Virtual.
- **mês:** os doze meses são gerados pelo próprio script, pois não existem como entidade no modelo relacional.
- **ano:** obtido pelos anos distintos das datas de venda (tb010_012_data). A chave primária é o próprio ano e, na base atual, existe apenas 2025.

Em relação ao tempo, portanto, o mês foi gerado e o ano foi extraído das datas existentes no relacional. A dimensão produto, prevista na primeira versão do modelo, foi retirada por não ser utilizada por nenhuma das consultas gerenciais atuais e, por isso, não consta nas Figuras 3 e 4 nem na carga.

4.4 - Carga da tabela fato

Seguindo a orientação de realizar o agrupamento na ETL, e não na análise, as agregações são feitas nas consultas de extração, por meio de SUM e GROUP BY no SQL Server. O valor de cada venda é calculado como quantidade * valor unitário (tb010_012_quantidade * tb010_012_valor_unitario). O resultado é inserido na fato_venda com as chaves estrangeiras das dimensões que não fazem parte do agrupamento preenchidas com NULL, representando "todas as opções" daquela dimensão, como no exemplo de tabela fato apresentado em aula. Assim, cada linha

da fato representa um nível de agregação, e as consultas gerenciais precisam apenas de filtros WHERE sobre as chaves nulas e não nulas.

Não foi utilizado GROUP BY ... WITH ROLLUP ou WITH CUBE. O ROLLUP gera apenas subtotais hierárquicos, e o CUBE gera todas as combinações possíveis entre as dimensões, com total de linhas igual ao produto de (n+1) de cada dimensão. Com oito dimensões, o CUBE resultaria em $2^8 = 256$ níveis de agrupamento, a maioria sem utilidade para o negócio. Por isso, foram carregados apenas os agrupamentos necessários para responder às perguntas gerenciais, conforme o quadro a seguir.

Carga (função)	Pergunta	Dimensões
carregar_fato_tipo_categoria	Quantidade de vendas por tipo e categoria	tipo, categoria
carregar_fato_funcionario_mes_ano	Valor das vendas por funcionário, com hierarquia de tempo	funcionário, mês, ano
carregar_fato_funcionario	Volume total de vendas por funcionário	funcionário
carregar_fato_funcionario_loja	Atendimentos por funcionário e localidade	funcionário, loja
carregar_fato_cliente_mes_ano	Valor das últimas vendas realizadas por cliente	cliente, mês, ano
carregar_fato_cliente_canal_mes_ano	Clientes que mais compraram na loja virtual, com valor acumulado por período	cliente, canal, mês, ano

4.5 - Controle de transações e execução

A ordem de execução respeita as dependências do modelo: primeiro são carregadas todas as dimensões e, depois, os seis agrupamentos da tabela fato, que dependem das chaves geradas nas dimensões. A conexão com o MySQL é feita com autocommit desativado, e cada etapa é confirmada (commit) ao seu final. Caso ocorra algum erro, é executado o rollback da etapa em andamento, a mensagem de erro é exibida e a carga é interrompida. As conexões com os dois bancos são encerradas ao final da execução, com ou sem erro. Ao término de cada agrupamento, o script informa a quantidade de linhas inseridas, o que permite conferir a carga.

O código completo do script dw.py está no Apêndice A. O script não limpa as tabelas antes da carga, para reexecutá-lo é necessário esvaziar o banco dw_vendas antes.

5 - Frequência de Carga

Como a tabela fato guarda agregações pré-calculadas, cada carga recalcula os agrupamentos. Seria melhor uma carga completa diária (limpar a fato_venda e recarregar dimensões e agrupamentos), executada a noite, após o fechamento das lojas, para não competir com o processamento transacional do BD_VAREJO. Como as análises são gerenciais e não exigem tempo real, a diferença de um dia é aceitável. Uma carga adicional ao fim de cada mês garante o fechamento dos totais mensais. Caso o volume de vendas cresça, a carga pode passar a ser incremental, recalculando apenas os agrupamentos afetados pelo período novo.

6 - Hardware e SGBD Recomendados

O SGBD escolhido para o BDM é o MySQL 8, em arquitetura ROLAP (armazenamento e processamento em SGBD relacional). Escolhemos a ferramenta pela ausência de custo de licenciamento (edição Community), pelo suporte a SQL e a chaves estrangeiras e pela independência em relação ao SQL Server de origem, mantendo separados os ambientes OLTP e OLAP. Para o volume atual (110 vendas e centenas de linhas na fato), um servidor simples é suficiente. Com o crescimento dos dados, seria melhor aumentar memória e armazenamento, criar índices nas chaves estrangeiras da fato, particionar a fato por ano e manter rotina de backup do BDM.

7 - Análise Gerencial

As consultas a seguir respondem às perguntas gerenciais definidas para o BDM. Nenhuma delas realiza agregação (GROUP BY) no momento da consulta: cada uma seleciona, por meio de filtros WHERE sobre chaves nulas e não nulas, o nível de agregação já calculado na carga. As consultas foram executadas no banco dw_vendas.

7.1 - Quantidade de vendas por tipo e categoria

```
SELECT t.nome tipo AS tipo, c.nome categoria AS categoria, func.nome completo AS funcionario, cli.nome AS cliente, CONCAT(l.pk loja, ' - ', l.cidade, '/', l.uf) AS loja, cn.nome canal AS canal, m.nome_mes AS mes, a.pk_ano AS ano, f.quantidade FROM fato_venda f LEFT JOIN tipo t ON f.fk_tipo = t.pk_tipo LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria LEFT JOIN funcionario func ON f.fk_funcionario = func.pk_funcionario LEFT JOIN cliente cli ON f.fk_cliente = cli.pk_cliente LEFT JOIN loja l ON f.fk_loja = l.pk_loja LEFT JOIN canal cn ON f.fk_canal = cn.pk_canal LEFT JOIN mes m ON f.fk_mes = m.pk_mes LEFT JOIN ano a ON f.fk_ano = a.pk_ano WHERE f.fk_tipo IS NOT NULL AND f.fk_categoria IS NOT NULL AND f.fk_funcionario IS NULL AND f.fk_cliente IS NULL AND f.fk_loja IS NULL AND f.fk_canal IS NULL AND f.fk_mes IS NULL AND f.fk_ano IS NULL ORDER BY t.nome_tipo, c.nome_categoria;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	quantidade
Alimento	Alimentos Perecíveis	NULL	NULL	NULL	NULL	NULL	NULL	51
Eleto	Eletrodomésticos	NULL	NULL	NULL	NULL	NULL	NULL	22
Eleto	Eletrônicos	NULL	NULL	NULL	NULL	NULL	NULL	14
Eleto	Mídias	NULL	NULL	NULL	NULL	NULL	NULL	17
Vestuario	Roupas Femininas	NULL	NULL	NULL	NULL	NULL	NULL	28
Vestuario	Roupas Infantis	NULL	NULL	NULL	NULL	NULL	NULL	9
Vestuario	Roupas Masculinas	NULL	NULL	NULL	NULL	NULL	NULL	29

7.2 - Valor das vendas por funcionário, com hierarquia de tempo (mês e ano)

```
SELECT t.nome_tipo AS tipo, c.nome_categoria AS categoria, func.nome_completo AS funcionario,
cli.nome AS cliente, CONCAT(l.pk_loja, ' - ', l.cidade, '/', l.uf) AS loja, cn.nome_canal AS canal,
m.nome_mes AS mes, a.pk_ano AS ano, f.valor FROM fato_venda f LEFT JOIN tipo t ON
f.fk_tipo = t.pk_tipo LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria LEFT JOIN
funcionario func ON f.fk_funcionario = func.pk_funcionario LEFT JOIN cliente cli ON
f.fk_cliente = cli.pk_cliente LEFT JOIN loja l ON f.fk_loja = l.pk_loja LEFT JOIN canal cn ON
f.fk_canal = cn.pk_canal LEFT JOIN mes m ON f.fk_mes = m.pk_mes LEFT JOIN ano a ON f.fk_ano =
a.pk_ano WHERE f.fk_funcionario IS NOT NULL AND f.fk_mes IS NOT NULL AND f.fk_ano IS NOT NULL
AND f.fk_tipo IS NULL AND f.fk_categoria IS NULL AND f.fk_cliente IS NULL AND f.fk_loja IS
NULL AND f.fk_canal IS NULL ORDER BY func.nome_completo, a.pk_ano, m.pk_mes;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	valor
NULL	NULL	Funcionário 01	NULL	NULL	NULL	Fevereiro	2025	407.94
NULL	NULL	Funcionário 01	NULL	NULL	NULL	Agosto	2025	3457.86
NULL	NULL	Funcionário 02	NULL	NULL	NULL	Março	2025	240.24
NULL	NULL	Funcionário 02	NULL	NULL	NULL	Setembro	2025	2275.16
NULL	NULL	Funcionário 03	NULL	NULL	NULL	Março	2025	18.38
NULL	NULL	Funcionário 03	NULL	NULL	NULL	Abril	2025	3567.60
NULL	NULL	Funcionário 04	NULL	NULL	NULL	Março	2025	20.10
NULL	NULL	Funcionário 04	NULL	NULL	NULL	Abril	2025	1600.00
NULL	NULL	Funcionário 04	NULL	NULL	NULL	Maio	2025	95.20
NULL	NULL	Funcionário 05	NULL	NULL	NULL	Março	2025	83.20
NULL	NULL	Funcionário 05	NULL	NULL	NULL	Abril	2025	574.40
NULL	NULL	Funcionário 05	NULL	NULL	NULL	Junho	2025	50.32
NULL	NULL	Funcionário 06	NULL	NULL	NULL	Janeiro	2025	37.18
NULL	NULL	Funcionário 06	NULL	NULL	NULL	Março	2025	53.54
NULL	NULL	Funcionário 06	NULL	NULL	NULL	Abril	2025	577.79
NULL	NULL	Funcionário 07	NULL	NULL	NULL	Março	2025	94.81
NULL	NULL	Funcionário 07	NULL	NULL	NULL	Abril	2025	895.78
NULL	NULL	Funcionário 08	NULL	NULL	NULL	Fevereiro	2025	17.31
NULL	NULL	Funcionário 08	NULL	NULL	NULL	Março	2025	52.88
NULL	NULL	Funcionário 08	NULL	NULL	NULL	Abril	2025	737.79
NULL	NULL	Funcionário 09	NULL	NULL	NULL	Março	2025	37.58
NULL	NULL	Funcionário 09	NULL	NULL	NULL	Abril	2025	24.00
NULL	NULL	Funcionário 09	NULL	NULL	NULL	Maio	2025	190.36
NULL	NULL	Funcionário 10	NULL	NULL	NULL	Março	2025	51.14
NULL	NULL	Funcionário 10	NULL	NULL	NULL	Abril	2025	27.20
NULL	NULL	Funcionário 10	NULL	NULL	NULL	Junho	2025	122.38
NULL	NULL	Funcionário 11	NULL	NULL	NULL	Março	2025	104.75
NULL	NULL	Funcionário 11	NULL	NULL	NULL	Abril	2025	35.20
NULL	NULL	Funcionário 12	NULL	NULL	NULL	Março	2025	148.18
NULL	NULL	Funcionário 12	NULL	NULL	NULL	Abril	2025	99.20
NULL	NULL	Funcionário 13	NULL	NULL	NULL	Março	2025	79.55
NULL	NULL	Funcionário 14	NULL	NULL	NULL	Março	2025	89.80
NULL	NULL	Funcionário 14	NULL	NULL	NULL	Abril	2025	86.40
NULL	NULL	Funcionário 15	NULL	NULL	NULL	Março	2025	83.11
NULL	NULL	Funcionário 15	NULL	NULL	NULL	Abril	2025	52.80
NULL	NULL	Funcionário 16	NULL	NULL	NULL	Março	2025	61.26
NULL	NULL	Funcionário 17	NULL	NULL	NULL	Março	2025	65.50
NULL	NULL	Funcionário 17	NULL	NULL	NULL	Abril	2025	43.20
NULL	NULL	Funcionário 18	NULL	NULL	NULL	Março	2025	95.94

NULL	NULL	Funcionário 18	NULL	NULL	NULL	Abril	2025	52.80
NULL	NULL	Funcionário 19	NULL	NULL	NULL	Março	2025	46.80
NULL	NULL	Funcionário 20	NULL	NULL	NULL	Janeiro	2025	1.10
NULL	NULL	Funcionário 20	NULL	NULL	NULL	Março	2025	1685.43
NULL	NULL	Funcionário 21	NULL	NULL	NULL	Fevereiro	2025	3.30
NULL	NULL	Funcionário 21	NULL	NULL	NULL	Março	2025	3094.76
NULL	NULL	Funcionário 22	NULL	NULL	NULL	Março	2025	15.12
NULL	NULL	Funcionário 23	NULL	NULL	NULL	Março	2025	32.84
NULL	NULL	Funcionário 23	NULL	NULL	NULL	Abril	2025	4.74
NULL	NULL	Funcionário 24	NULL	NULL	NULL	Março	2025	617.82
NULL	NULL	Funcionário 24	NULL	NULL	NULL	Mai	2025	1.63
NULL	NULL	Funcionário 25	NULL	NULL	NULL	Março	2025	456.53
NULL	NULL	Funcionário 25	NULL	NULL	NULL	Junho	2025	2.20
NULL	NULL	Funcionário 26	NULL	NULL	NULL	Janeiro	2025	27.18
NULL	NULL	Funcionário 26	NULL	NULL	NULL	Julho	2025	8.88
NULL	NULL	Funcionário 28	NULL	NULL	NULL	Março	2025	715.27
NULL	NULL	Funcionário 29	NULL	NULL	NULL	Janeiro	2025	35.34
NULL	NULL	Funcionário 29	NULL	NULL	NULL	Março	2025	732.14
NULL	NULL	Funcionário 32	NULL	NULL	NULL	Fevereiro	2025	27.18
NULL	NULL	Funcionário 32	NULL	NULL	NULL	Março	2025	340.91
NULL	NULL	Funcionário 33	NULL	NULL	NULL	Março	2025	465.36
NULL	NULL	Funcionário 36	NULL	NULL	NULL	Março	2025	1968.94
NULL	NULL	Funcionário 36	NULL	NULL	NULL	Abril	2025	108.76
NULL	NULL	Funcionário 37	NULL	NULL	NULL	Março	2025	1961.82
NULL	NULL	Funcionário 37	NULL	NULL	NULL	Mai	2025	81.58
NULL	NULL	Funcionário 40	NULL	NULL	NULL	Março	2025	18.54
NULL	NULL	Funcionário 40	NULL	NULL	NULL	Junho	2025	14.38

7.3 - Volume total de vendas por funcionário

```
SELECT t.nome_tipo AS tipo, c.nome_categoria AS categoria, func.nome_completo AS funcionario,
       cli.nome AS cliente, CONCAT(l.pk_loja, ' - ', l.cidade, '/', l.uf) AS loja,
       cn.nome_canal AS canal, m.nome_mes AS mes, a.pk_ano AS ano, f.quantidade, f.valor
FROM fato_venda f
LEFT JOIN tipo t ON f.fk_tipo = t.pk_tipo
LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria
LEFT JOIN funcionario func ON f.fk_funcionario = func.pk_funcionario
LEFT JOIN cliente cli ON f.fk_cliente = cli.pk_cliente
LEFT JOIN loja l ON f.fk_loja = l.pk_loja
LEFT JOIN canal cn ON f.fk_canal = cn.pk_canal
LEFT JOIN mes m ON f.fk_mes = m.pk_mes
LEFT JOIN ano a ON f.fk_ano = a.pk_ano
WHERE f.fk_funcionario IS NOT NULL
      AND f.fk_tipo IS NULL AND f.fk_categoria IS NULL AND f.fk_cliente IS NULL
      AND f.fk_loja IS NULL AND f.fk_canal IS NULL AND f.fk_mes IS NULL AND f.fk_ano IS NULL
ORDER BY f.valor DESC;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	quantidade	valor
NULL	NULL	Funcionário 01	NULL	NULL	NULL	NULL	NULL	6	3865.80
NULL	NULL	Funcionário 03	NULL	NULL	NULL	NULL	NULL	5	3585.98
NULL	NULL	Funcionário 21	NULL	NULL	NULL	NULL	NULL	9	3098.06
NULL	NULL	Funcionário 02	NULL	NULL	NULL	NULL	NULL	10	2515.40
NULL	NULL	Funcionário 36	NULL	NULL	NULL	NULL	NULL	3	2077.70
NULL	NULL	Funcionário 37	NULL	NULL	NULL	NULL	NULL	3	2043.40
NULL	NULL	Funcionário 04	NULL	NULL	NULL	NULL	NULL	6	1715.30
NULL	NULL	Funcionário 20	NULL	NULL	NULL	NULL	NULL	4	1686.53
NULL	NULL	Funcionário 07	NULL	NULL	NULL	NULL	NULL	9	990.59
NULL	NULL	Funcionário 08	NULL	NULL	NULL	NULL	NULL	4	807.98
NULL	NULL	Funcionário 29	NULL	NULL	NULL	NULL	NULL	2	767.48
NULL	NULL	Funcionário 28	NULL	NULL	NULL	NULL	NULL	3	715.27
NULL	NULL	Funcionário 05	NULL	NULL	NULL	NULL	NULL	10	707.92
NULL	NULL	Funcionário 06	NULL	NULL	NULL	NULL	NULL	5	668.51
NULL	NULL	Funcionário 24	NULL	NULL	NULL	NULL	NULL	5	619.45
NULL	NULL	Funcionário 33	NULL	NULL	NULL	NULL	NULL	4	465.36
NULL	NULL	Funcionário 25	NULL	NULL	NULL	NULL	NULL	7	458.73
NULL	NULL	Funcionário 32	NULL	NULL	NULL	NULL	NULL	2	368.09
NULL	NULL	Funcionário 09	NULL	NULL	NULL	NULL	NULL	6	251.94
NULL	NULL	Funcionário 12	NULL	NULL	NULL	NULL	NULL	8	247.38
NULL	NULL	Funcionário 10	NULL	NULL	NULL	NULL	NULL	4	200.72
NULL	NULL	Funcionário 14	NULL	NULL	NULL	NULL	NULL	6	176.20
NULL	NULL	Funcionário 18	NULL	NULL	NULL	NULL	NULL	6	148.74
NULL	NULL	Funcionário 11	NULL	NULL	NULL	NULL	NULL	6	139.95
NULL	NULL	Funcionário 15	NULL	NULL	NULL	NULL	NULL	5	135.91
NULL	NULL	Funcionário 17	NULL	NULL	NULL	NULL	NULL	5	108.70
NULL	NULL	Funcionário 13	NULL	NULL	NULL	NULL	NULL	6	79.55
NULL	NULL	Funcionário 16	NULL	NULL	NULL	NULL	NULL	3	61.26
NULL	NULL	Funcionário 19	NULL	NULL	NULL	NULL	NULL	3	46.80
NULL	NULL	Funcionário 23	NULL	NULL	NULL	NULL	NULL	5	37.58
NULL	NULL	Funcionário 26	NULL	NULL	NULL	NULL	NULL	5	36.06
NULL	NULL	Funcionário 40	NULL	NULL	NULL	NULL	NULL	2	32.92
NULL	NULL	Funcionário 22	NULL	NULL	NULL	NULL	NULL	3	15.12

7.4 - Atendimentos por funcionário e localidade (loja)

```
SELECT t.nome_tipo AS tipo, c.nome_categoria AS categoria, func.nome_completo AS funcionario,
       cli.nome AS cliente, CONCAT(l.pk_loja, ' - ', l.cidade, '/', l.uf) AS loja,
       cn.nome_canal AS canal, m.nome_mes AS mes, a.pk_ano AS ano, f.quantidade
FROM fato_venda f
LEFT JOIN tipo t ON f.fk_tipo = t.pk_tipo
LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria
LEFT JOIN funcionario func ON f.fk_funcionario = func.pk_funcionario
LEFT JOIN cliente cli ON f.fk_cliente = cli.pk_cliente
LEFT JOIN loja l ON f.fk_loja = l.pk_loja
LEFT JOIN canal cn ON f.fk_canal = cn.pk_canal
LEFT JOIN mes m ON f.fk_mes = m.pk_mes
LEFT JOIN ano a ON f.fk_ano = a.pk_ano
WHERE f.fk_funcionario IS NOT NULL AND f.fk_loja IS NOT NULL
      AND f.fk_tipo IS NULL AND f.fk_categoria IS NULL AND f.fk_cliente IS NULL
      AND f.fk_canal IS NULL AND f.fk_mes IS NULL AND f.fk_ano IS NULL
ORDER BY l.cidade, func.nome_completo;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	quantidade
NULL	NULL	Funcionário 02	NULL	2 - Belo Horizonte/MG	NULL	NULL	NULL	10
NULL	NULL	Funcionário 07	NULL	2 - Belo Horizonte/MG	NULL	NULL	NULL	9
NULL	NULL	Funcionário 12	NULL	2 - Belo Horizonte/MG	NULL	NULL	NULL	8
NULL	NULL	Funcionário 22	NULL	2 - Belo Horizonte/MG	NULL	NULL	NULL	3
NULL	NULL	Funcionário 01	NULL	1 - Curitiba/PR	NULL	NULL	NULL	6
NULL	NULL	Funcionário 05	NULL	5 - Curitiba/PR	NULL	NULL	NULL	10
NULL	NULL	Funcionário 06	NULL	1 - Curitiba/PR	NULL	NULL	NULL	5
NULL	NULL	Funcionário 10	NULL	5 - Curitiba/PR	NULL	NULL	NULL	4
NULL	NULL	Funcionário 11	NULL	1 - Curitiba/PR	NULL	NULL	NULL	6
NULL	NULL	Funcionário 15	NULL	5 - Curitiba/PR	NULL	NULL	NULL	5
NULL	NULL	Funcionário 16	NULL	1 - Curitiba/PR	NULL	NULL	NULL	3
NULL	NULL	Funcionário 17	NULL	1 - Curitiba/PR	NULL	NULL	NULL	5
NULL	NULL	Funcionário 18	NULL	1 - Curitiba/PR	NULL	NULL	NULL	6
NULL	NULL	Funcionário 19	NULL	1 - Curitiba/PR	NULL	NULL	NULL	3
NULL	NULL	Funcionário 20	NULL	1 - Curitiba/PR	NULL	NULL	NULL	4
NULL	NULL	Funcionário 21	NULL	1 - Curitiba/PR	NULL	NULL	NULL	9
NULL	NULL	Funcionário 32	NULL	5 - Curitiba/PR	NULL	NULL	NULL	2
NULL	NULL	Funcionário 33	NULL	1 - Curitiba/PR	NULL	NULL	NULL	4
NULL	NULL	Funcionário 36	NULL	1 - Curitiba/PR	NULL	NULL	NULL	3
NULL	NULL	Funcionário 37	NULL	1 - Curitiba/PR	NULL	NULL	NULL	3
NULL	NULL	Funcionário 40	NULL	1 - Curitiba/PR	NULL	NULL	NULL	2
NULL	NULL	Funcionário 03	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	5
NULL	NULL	Funcionário 08	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	4
NULL	NULL	Funcionário 13	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	6
NULL	NULL	Funcionário 23	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	5
NULL	NULL	Funcionário 24	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	5
NULL	NULL	Funcionário 25	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	7
NULL	NULL	Funcionário 26	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	5
NULL	NULL	Funcionário 28	NULL	3 - Porto Alegre/RS	NULL	NULL	NULL	3
NULL	NULL	Funcionário 04	NULL	4 - São Paulo/SP	NULL	NULL	NULL	6
NULL	NULL	Funcionário 09	NULL	4 - São Paulo/SP	NULL	NULL	NULL	6
NULL	NULL	Funcionário 14	NULL	4 - São Paulo/SP	NULL	NULL	NULL	6
NULL	NULL	Funcionário 29	NULL	4 - São Paulo/SP	NULL	NULL	NULL	2

7.5 - Valor das últimas vendas realizadas por cliente

```
SELECT t.nome_tipo AS tipo, c.nome_categoria AS categoria, func.nome_completo AS funcionario,
       cli.nome AS cliente, CONCAT(l.pk_loja, ' - ', l.cidade, '/', l.uf) AS loja,
       cn.nome_canal AS canal, m.nome_mes AS mes, a.pk_ano AS ano, f.valor
FROM fato_venda f
LEFT JOIN tipo t ON f.fk_tipo = t.pk_tipo
LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria
LEFT JOIN funcionario func ON f.fk_funcionario = func.pk_funcionario
LEFT JOIN cliente cli ON f.fk_cliente = cli.pk_cliente
LEFT JOIN loja l ON f.fk_loja = l.pk_loja
LEFT JOIN canal cn ON f.fk_canal = cn.pk_canal
LEFT JOIN mes m ON f.fk_mes = m.pk_mes
LEFT JOIN ano a ON f.fk_ano = a.pk_ano
WHERE f.fk_cliente IS NOT NULL AND f.fk_mes IS NOT NULL AND f.fk_ano IS NOT NULL
      AND f.fk_tipo IS NULL AND f.fk_categoria IS NULL AND f.fk_funcionario IS NULL
      AND f.fk_loja IS NULL AND f.fk_canal IS NULL
ORDER BY cli.nome, a.pk_ano, m.pk_mes;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	valor
NULL	NULL	NULL	NOME Teste 01	NULL	NULL	Março	2025	68.10
NULL	NULL	NULL	NOME Teste 02	NULL	NULL	Março	2025	106.14
NULL	NULL	NULL	NOME Teste 03	NULL	NULL	Março	2025	66.34
NULL	NULL	NULL	NOME Teste 04	NULL	NULL	Março	2025	67.82
NULL	NULL	NULL	NOME Teste 05	NULL	NULL	Março	2025	74.55
NULL	NULL	NULL	NOME Teste 06	NULL	NULL	Março	2025	106.36
NULL	NULL	NULL	NOME Teste 07	NULL	NULL	Janeiro	2025	27.18
NULL	NULL	NULL	NOME Teste 07	NULL	NULL	Março	2025	88.30
NULL	NULL	NULL	NOME Teste 08	NULL	NULL	Fevereiro	2025	407.94
NULL	NULL	NULL	NOME Teste 08	NULL	NULL	Março	2025	16.04
NULL	NULL	NULL	NOME Teste 09	NULL	NULL	Março	2025	194.72
NULL	NULL	NULL	NOME Teste 10	NULL	NULL	Março	2025	20.98
NULL	NULL	NULL	NOME Teste 10	NULL	NULL	Abril	2025	47.60
NULL	NULL	NULL	NOME Teste 11	NULL	NULL	Março	2025	19.94
NULL	NULL	NULL	NOME Teste 11	NULL	NULL	Maio	2025	95.20
NULL	NULL	NULL	NOME Teste 12	NULL	NULL	Março	2025	33.86
NULL	NULL	NULL	NOME Teste 12	NULL	NULL	Junho	2025	50.32
NULL	NULL	NULL	NOME Teste 13	NULL	NULL	Janeiro	2025	35.34
NULL	NULL	NULL	NOME Teste 13	NULL	NULL	Março	2025	116.67
NULL	NULL	NULL	NOME Teste 14	NULL	NULL	Fevereiro	2025	27.18
NULL	NULL	NULL	NOME Teste 14	NULL	NULL	Março	2025	190.51
NULL	NULL	NULL	NOME Teste 14	NULL	NULL	Maio	2025	81.58
NULL	NULL	NULL	NOME Teste 15	NULL	NULL	Março	2025	197.63
NULL	NULL	NULL	NOME Teste 15	NULL	NULL	Junho	2025	14.38
NULL	NULL	NULL	NOME Teste 16	NULL	NULL	Março	2025	485.80
NULL	NULL	NULL	NOME Teste 16	NULL	NULL	Abril	2025	108.76
NULL	NULL	NULL	NOME Teste 17	NULL	NULL	Março	2025	1970.49
NULL	NULL	NULL	NOME Teste 18	NULL	NULL	Março	2025	1963.24
NULL	NULL	NULL	NOME Teste 19	NULL	NULL	Março	2025	22.98
NULL	NULL	NULL	NOME Teste 20	NULL	NULL	Março	2025	20.42
NULL	NULL	NULL	NOME Teste 21	NULL	NULL	Março	2025	77.50
NULL	NULL	NULL	NOME Teste 22	NULL	NULL	Janeiro	2025	1.10
NULL	NULL	NULL	NOME Teste 22	NULL	NULL	Março	2025	17.28
NULL	NULL	NULL	NOME Teste 23	NULL	NULL	Fevereiro	2025	3.30
NULL	NULL	NULL	NOME Teste 23	NULL	NULL	Março	2025	55.74
NULL	NULL	NULL	NOME Teste 24	NULL	NULL	Março	2025	18.96
NULL	NULL	NULL	NOME Teste 25	NULL	NULL	Janeiro	2025	37.18
NULL	NULL	NULL	NOME Teste 25	NULL	NULL	Abril	2025	4.74
NULL	NULL	NULL	NOME Teste 26	NULL	NULL	Fevereiro	2025	17.31

NULL	NULL	NULL	NOME Teste 26	NULL	NULL	Maio	2025	1.63
NULL	NULL	NULL	NOME Teste 27	NULL	NULL	Março	2025	18.61
NULL	NULL	NULL	NOME Teste 27	NULL	NULL	Junho	2025	2.20
NULL	NULL	NULL	NOME Teste 28	NULL	NULL	Abril	2025	1.38
NULL	NULL	NULL	NOME Teste 28	NULL	NULL	Julho	2025	8.88
NULL	NULL	NULL	NOME Teste 29	NULL	NULL	Maio	2025	190.36
NULL	NULL	NULL	NOME Teste 29	NULL	NULL	Agosto	2025	3457.86
NULL	NULL	NULL	NOME Teste 30	NULL	NULL	Junho	2025	122.38
NULL	NULL	NULL	NOME Teste 30	NULL	NULL	Setembro	2025	2275.16
NULL	NULL	NULL	NOME Teste 31	NULL	NULL	Março	2025	59.88
NULL	NULL	NULL	NOME Teste 31	NULL	NULL	Abril	2025	3520.00
NULL	NULL	NULL	NOME Teste 32	NULL	NULL	Março	2025	29.97
NULL	NULL	NULL	NOME Teste 32	NULL	NULL	Abril	2025	1600.00
NULL	NULL	NULL	NOME Teste 33	NULL	NULL	Março	2025	59.96
NULL	NULL	NULL	NOME Teste 33	NULL	NULL	Abril	2025	574.40
NULL	NULL	NULL	NOME Teste 34	NULL	NULL	Março	2025	47.60
NULL	NULL	NULL	NOME Teste 34	NULL	NULL	Abril	2025	577.79
NULL	NULL	NULL	NOME Teste 35	NULL	NULL	Março	2025	73.44
NULL	NULL	NULL	NOME Teste 35	NULL	NULL	Abril	2025	894.40
NULL	NULL	NULL	NOME Teste 36	NULL	NULL	Abril	2025	737.79
NULL	NULL	NULL	NOME Teste 37	NULL	NULL	Abril	2025	24.00
NULL	NULL	NULL	NOME Teste 38	NULL	NULL	Março	2025	51.68
NULL	NULL	NULL	NOME Teste 38	NULL	NULL	Abril	2025	27.20
NULL	NULL	NULL	NOME Teste 39	NULL	NULL	Março	2025	59.84
NULL	NULL	NULL	NOME Teste 39	NULL	NULL	Abril	2025	35.20
NULL	NULL	NULL	NOME Teste 40	NULL	NULL	Março	2025	47.60
NULL	NULL	NULL	NOME Teste 40	NULL	NULL	Abril	2025	99.20
NULL	NULL	NULL	NOME Teste 41	NULL	NULL	Março	2025	76.38
NULL	NULL	NULL	NOME Teste 41	NULL	NULL	Abril	2025	86.40
NULL	NULL	NULL	NOME Teste 42	NULL	NULL	Março	2025	43.54
NULL	NULL	NULL	NOME Teste 42	NULL	NULL	Abril	2025	52.80
NULL	NULL	NULL	NOME Teste 43	NULL	NULL	Abril	2025	43.20
NULL	NULL	NULL	NOME Teste 44	NULL	NULL	Abril	2025	52.80
NULL	NULL	NULL	NOME Teste 45	NULL	NULL	Março	2025	1643.31
NULL	NULL	NULL	NOME Teste 46	NULL	NULL	Março	2025	2949.74
NULL	NULL	NULL	NOME Teste 47	NULL	NULL	Março	2025	406.62
NULL	NULL	NULL	NOME Teste 48	NULL	NULL	Março	2025	229.74
NULL	NULL	NULL	NOME Teste 49	NULL	NULL	Março	2025	660.91
NULL	NULL	NULL	NOME Teste 50	NULL	NULL	Março	2025	732.14
NULL	NULL	NULL	NOME Teste 51	NULL	NULL	Março	2025	340.91

7.6 - Clientes que mais compraram na loja virtual, com valor acumulado por período

```
SELECT t.nome_tipo AS tipo, c.nome_categoria AS categoria, func.nome_completo AS funcionario,
       cli.nome AS cliente, CONCAT(l.pk_loja, ' - ', l.cidade, '/', l.uf) AS loja,
       cn.nome_canal AS canal, m.nome_mes AS mes, a.pk_ano AS ano, f.valor
FROM fato_venda f
LEFT JOIN tipo t ON f.fk_tipo = t.pk_tipo
LEFT JOIN categoria c ON f.fk_categoria = c.pk_categoria
LEFT JOIN funcionario func ON f.fk_funcionario = func.pk_funcionario
LEFT JOIN cliente cli ON f.fk_cliente = cli.pk_cliente
LEFT JOIN loja l ON f.fk_loja = l.pk_loja
LEFT JOIN canal cn ON f.fk_canal = cn.pk_canal
LEFT JOIN mes m ON f.fk_mes = m.pk_mes
LEFT JOIN ano a ON f.fk_ano = a.pk_ano
WHERE f.fk_cliente IS NOT NULL AND f.fk_canal IS NOT NULL
      AND f.fk_mes IS NOT NULL AND f.fk_ano IS NOT NULL
      AND f.fk_tipo IS NULL AND f.fk_categoria IS NULL AND f.fk_funcionario IS NULL
      AND f.fk_loja IS NULL
ORDER BY f.valor DESC;
```

tipo	categoria	funcionario	cliente	loja	canal	mes	ano	valor
NULL	NULL	NULL	NOME Teste 31	NULL	Loja Virtual	Abril	2025	3520.00
NULL	NULL	NULL	NOME Teste 29	NULL	Loja Virtual	Agosto	2025	3457.86
NULL	NULL	NULL	NOME Teste 46	NULL	Loja Virtual	Março	2025	2949.74
NULL	NULL	NULL	NOME Teste 30	NULL	Loja Virtual	Setembro	2025	2275.16
NULL	NULL	NULL	NOME Teste 17	NULL	Loja Virtual	Março	2025	1970.49
NULL	NULL	NULL	NOME Teste 18	NULL	Loja Virtual	Março	2025	1963.24
NULL	NULL	NULL	NOME Teste 45	NULL	Loja Virtual	Março	2025	1643.31
NULL	NULL	NULL	NOME Teste 32	NULL	Loja Virtual	Abril	2025	1600.00
NULL	NULL	NULL	NOME Teste 35	NULL	Loja Virtual	Abril	2025	894.40
NULL	NULL	NULL	NOME Teste 36	NULL	Loja Virtual	Abril	2025	737.79
NULL	NULL	NULL	NOME Teste 50	NULL	Loja Virtual	Março	2025	732.14
NULL	NULL	NULL	NOME Teste 49	NULL	Loja Virtual	Março	2025	660.91
NULL	NULL	NULL	NOME Teste 34	NULL	Loja Virtual	Abril	2025	577.79
NULL	NULL	NULL	NOME Teste 33	NULL	Loja Virtual	Abril	2025	574.40
NULL	NULL	NULL	NOME Teste 16	NULL	Loja Virtual	Março	2025	485.80
NULL	NULL	NULL	NOME Teste 08	NULL	Loja Virtual	Fevereiro	2025	407.94
NULL	NULL	NULL	NOME Teste 47	NULL	Loja Virtual	Março	2025	406.62
NULL	NULL	NULL	NOME Teste 51	NULL	Loja Virtual	Março	2025	340.91
NULL	NULL	NULL	NOME Teste 48	NULL	Loja Virtual	Março	2025	229.74
NULL	NULL	NULL	NOME Teste 15	NULL	Loja Virtual	Março	2025	197.63
NULL	NULL	NULL	NOME Teste 09	NULL	Loja Virtual	Março	2025	194.72
NULL	NULL	NULL	NOME Teste 14	NULL	Loja Virtual	Março	2025	190.51
NULL	NULL	NULL	NOME Teste 29	NULL	Loja Virtual	Maio	2025	190.36
NULL	NULL	NULL	NOME Teste 30	NULL	Loja Virtual	Junho	2025	122.38
NULL	NULL	NULL	NOME Teste 13	NULL	Loja Virtual	Março	2025	116.67
NULL	NULL	NULL	NOME Teste 16	NULL	Loja Virtual	Abril	2025	108.76
NULL	NULL	NULL	NOME Teste 06	NULL	Loja Virtual	Março	2025	106.36
NULL	NULL	NULL	NOME Teste 02	NULL	Loja Virtual	Março	2025	106.14
NULL	NULL	NULL	NOME Teste 40	NULL	Loja Virtual	Abril	2025	99.20
NULL	NULL	NULL	NOME Teste 11	NULL	Loja Virtual	Maio	2025	95.20
NULL	NULL	NULL	NOME Teste 07	NULL	Loja Virtual	Março	2025	88.30
NULL	NULL	NULL	NOME Teste 41	NULL	Loja Virtual	Abril	2025	86.40
NULL	NULL	NULL	NOME Teste 14	NULL	Loja Virtual	Maio	2025	81.58
NULL	NULL	NULL	NOME Teste 21	NULL	Loja Virtual	Março	2025	77.50
NULL	NULL	NULL	NOME Teste 41	NULL	Loja Virtual	Março	2025	76.38
NULL	NULL	NULL	NOME Teste 05	NULL	Loja Virtual	Março	2025	74.55
NULL	NULL	NULL	NOME Teste 35	NULL	Loja Virtual	Março	2025	73.44
NULL	NULL	NULL	NOME Teste 01	NULL	Loja Virtual	Março	2025	68.10
NULL	NULL	NULL	NOME Teste 04	NULL	Loja Virtual	Março	2025	67.82
NULL	NULL	NULL	NOME Teste 03	NULL	Loja Virtual	Março	2025	66.34
NULL	NULL	NULL	NOME Teste 33	NULL	Loja Virtual	Março	2025	59.96

NULL	NULL	NULL	NOME Teste 31	NULL	Loja Virtual	Margo	2025	59.88
NULL	NULL	NULL	NOME Teste 39	NULL	Loja Virtual	Margo	2025	59.84
NULL	NULL	NULL	NOME Teste 23	NULL	Loja Virtual	Margo	2025	55.74
NULL	NULL	NULL	NOME Teste 42	NULL	Loja Virtual	Abril	2025	52.80
NULL	NULL	NULL	NOME Teste 44	NULL	Loja Virtual	Abril	2025	52.80
NULL	NULL	NULL	NOME Teste 38	NULL	Loja Virtual	Margo	2025	51.68
NULL	NULL	NULL	NOME Teste 12	NULL	Loja Virtual	Junho	2025	50.32
NULL	NULL	NULL	NOME Teste 40	NULL	Loja Virtual	Margo	2025	47.60
NULL	NULL	NULL	NOME Teste 34	NULL	Loja Virtual	Margo	2025	47.60
NULL	NULL	NULL	NOME Teste 10	NULL	Loja Virtual	Abril	2025	47.60
NULL	NULL	NULL	NOME Teste 42	NULL	Loja Virtual	Margo	2025	43.54
NULL	NULL	NULL	NOME Teste 43	NULL	Loja Virtual	Abril	2025	43.20
NULL	NULL	NULL	NOME Teste 25	NULL	Loja Virtual	Janeiro	2025	37.18
NULL	NULL	NULL	NOME Teste 13	NULL	Loja Virtual	Janeiro	2025	35.34
NULL	NULL	NULL	NOME Teste 39	NULL	Loja Virtual	Abril	2025	35.20
NULL	NULL	NULL	NOME Teste 12	NULL	Loja Virtual	Margo	2025	33.86
NULL	NULL	NULL	NOME Teste 32	NULL	Loja Virtual	Margo	2025	29.97
NULL	NULL	NULL	NOME Teste 38	NULL	Loja Virtual	Abril	2025	27.20
NULL	NULL	NULL	NOME Teste 14	NULL	Loja Virtual	Fevereiro	2025	27.18
NULL	NULL	NULL	NOME Teste 07	NULL	Loja Virtual	Janeiro	2025	27.18
NULL	NULL	NULL	NOME Teste 37	NULL	Loja Virtual	Abril	2025	24.00
NULL	NULL	NULL	NOME Teste 19	NULL	Loja Virtual	Margo	2025	22.98
NULL	NULL	NULL	NOME Teste 10	NULL	Loja Virtual	Margo	2025	20.98
NULL	NULL	NULL	NOME Teste 20	NULL	Loja Virtual	Margo	2025	20.42
NULL	NULL	NULL	NOME Teste 11	NULL	Loja Virtual	Margo	2025	19.94
NULL	NULL	NULL	NOME Teste 24	NULL	Loja Virtual	Margo	2025	18.96
NULL	NULL	NULL	NOME Teste 27	NULL	Loja Virtual	Margo	2025	18.61
NULL	NULL	NULL	NOME Teste 26	NULL	Loja Virtual	Fevereiro	2025	17.31
NULL	NULL	NULL	NOME Teste 22	NULL	Loja Virtual	Margo	2025	17.28
NULL	NULL	NULL	NOME Teste 08	NULL	Loja Virtual	Margo	2025	16.04
NULL	NULL	NULL	NOME Teste 15	NULL	Loja Virtual	Junho	2025	14.38
NULL	NULL	NULL	NOME Teste 28	NULL	Loja Virtual	Julho	2025	8.88
NULL	NULL	NULL	NOME Teste 25	NULL	Loja Virtual	Abril	2025	4.74
NULL	NULL	NULL	NOME Teste 23	NULL	Loja Virtual	Fevereiro	2025	3.30
NULL	NULL	NULL	NOME Teste 27	NULL	Loja Virtual	Junho	2025	2.20
NULL	NULL	NULL	NOME Teste 26	NULL	Loja Virtual	Mai	2025	1.63
NULL	NULL	NULL	NOME Teste 28	NULL	Loja Virtual	Abril	2025	1.38
NULL	NULL	NULL	NOME Teste 22	NULL	Loja Virtual	Janeiro	2025	1.10

Como todas as vendas foram classificadas como Loja Virtual, a consulta retorna todos os clientes.

8 - Conclusão

Este trabalho propôs um Data Warehouse para a rede varejista, separando o ambiente operacional (BD_VAREJO, no SQL Server) do analítico (BDM, no MySQL). Foram documentadas as divergências entre o backup e o modelo original, definido o modelo conceitual e lógico em torno da fato_venda com oito dimensões e implementado um processo de ETL que trata a origem, calcula as medidas e carrega a fato com agregações pré-calculadas, em que o valor NULL representa "todas as opções" de uma dimensão. Com isso, as consultas gerenciais foram respondidas apenas com filtros WHERE, sem agregação em tempo de consulta, o que as torna simples e leves. Como limitações, o canal não diferencia as vendas (todos os clientes possuem login) e a dimensão funcionário possui linhas duplicadas para quem tem mais de um cargo. Em relação a melhorias, podem ser incluídas a dimensão produto, a fato de compras e novos agrupamentos conforme surjam novas perguntas de negócio.

Apêndice A – Código do ETL (dw.py) e DDL

A.1 - DDL

```
CREATE TABLE tipo (pk_tipo INT AUTO_INCREMENT, nome_tipo VARCHAR(20) NOT NULL, PRIMARY KEY (pk_tipo));
CREATE TABLE cliente (pk_cliente INT AUTO_INCREMENT, nome VARCHAR(255) NOT NULL, PRIMARY KEY (pk_cliente));
CREATE TABLE loja (pk_loja INT AUTO_INCREMENT, cidade VARCHAR(255) NOT NULL, uf VARCHAR(2), PRIMARY KEY (pk_loja));
CREATE TABLE mes (pk_mes INT AUTO_INCREMENT, nome_mes VARCHAR(20) NOT NULL, PRIMARY KEY (pk_mes));
CREATE TABLE categoria (pk_categoria INT AUTO_INCREMENT, nome_categoria VARCHAR(255) NOT NULL, PRIMARY KEY (pk_categoria));
CREATE TABLE funcionario (pk_funcionario INT AUTO_INCREMENT, nome_completo VARCHAR(255) NOT NULL, cargo VARCHAR(255), PRIMARY KEY (pk_funcionario));
CREATE TABLE canal (pk_canal INT AUTO_INCREMENT, nome_canal VARCHAR(20) NOT NULL, PRIMARY KEY (pk_canal));
CREATE TABLE ano (pk_ano INT, PRIMARY KEY (pk_ano));
CREATE TABLE fato_venda (
    fk_tipo INT, fk_categoria INT, fk_funcionario INT, fk_cliente INT,
    fk_loja INT, fk_canal INT, fk_mes INT, fk_ano INT,
    quantidade INT, valor DECIMAL(12, 2),
    CONSTRAINT fk_fato_tipo FOREIGN KEY (fk_tipo) REFERENCES tipo(pk_tipo),
    CONSTRAINT fk_fato_categoria FOREIGN KEY (fk_categoria) REFERENCES categoria(pk_categoria),
    CONSTRAINT fk_fato_funcionario FOREIGN KEY (fk_funcionario) REFERENCES funcionario(pk_funcionario),
    CONSTRAINT fk_fato_cliente FOREIGN KEY (fk_cliente) REFERENCES cliente(pk_cliente),
    CONSTRAINT fk_fato_loja FOREIGN KEY (fk_loja) REFERENCES loja(pk_loja),
    CONSTRAINT fk_fato_canal FOREIGN KEY (fk_canal) REFERENCES canal(pk_canal),
    CONSTRAINT fk_fato_mes FOREIGN KEY (fk_mes) REFERENCES mes(pk_mes),
    CONSTRAINT fk_fato_ano FOREIGN KEY (fk_ano) REFERENCES ano(pk_ano)
);
```

A.2 - Script dw.py

```
"""
Requisitos:
    pip install pyodbc pymysql
"""

import pyodbc
import pymysql

# -----
# 1. conexões
# editar com base na sua configuracao
# -----

def conectar_origem():
    return pyodbc.connect(
        "DRIVER={ODBC Driver 17 for SQL Server};"
        "SERVER=localhost\\SQLEXPRESS;"
        "DATABASE=ADS;"
        "Trusted_Connection=yes;"
    )

def conectar_destino():
    return pymysql.connect(
        host="localhost",
        user="root",
        password="*****",
        database="dw_vendas",
        autocommit=False, #
    )

# -----
# 2. carga das dimensões:
# 2.1 primeiro select pra pegar os dados na origem
# 2.2 insere os dados na tabela do mysql
# 2.3 volta um dicionario que associa o id original (tipo um cpf de cliente
#     ou matricula) com um novo id gerado no dw (no mysql)
# -----

def carregar_dim_tipo(conn_dest):
    """insere manualmente pq nao tem na fonte"""
    cur = conn_dest.cursor()
    mapa = {}
    for nome in ["Alimento", "Eletro", "Vestuario"]:
```

```

        cur.execute("INSERT INTO tipo (nome_tipo) VALUES (%s)", (nome,))
        mapa[nome] = cur.lastrowid
        conn_dest.commit()
        return mapa

def carregar_dim_categoria(conn_origem, conn_dest):
    # cursor p fazer as consultas no sqlserver
    cur_o = conn_origem.cursor()
    cur_o.execute("SELECT tb013_cod_categoria, tb013_descricao FROM tb013_categorias")
    linhas = cur_o.fetchall()

    # cursor p fazer os inserts no mysql
    cur_d = conn_dest.cursor()
    # esse mapa é p criar as chaves auto incrementais la no mysql
    mapa = {}
    for cod, descricao in linhas:
        cur_d.execute("INSERT INTO categoria (nome_categoria) VALUES (%s)", (descricao,))
        # captura o id criado la no mysql
        mapa[cod] = cur_d.lastrowid
    # salva para efetuar realmente apenas se nao tiver erros na execucao
    conn_dest.commit()
    return mapa

def carregar_dim_funcionario(conn_origem, conn_dest):
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT f.tb005_matricula, f.tb005_nome_completo, c.tb006_nome_cargo
        FROM tb005_funcionarios f
        LEFT JOIN tb005_006_funcionarios_cargos fc ON f.tb005_matricula = fc.tb005_matricula
        LEFT JOIN tb006_cargos c ON fc.tb006_cod_cargo = c.tb006_cod_cargo
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    mapa = {}
    for matricula, nome, cargo in linhas:
        cur_d.execute(
            "INSERT INTO funcionario (nome_completo, cargo) VALUES (%s, %s)",
            (nome, cargo),
        )
        mapa[matricula] = cur_d.lastrowid
    conn_dest.commit()
    return mapa

def carregar_dim_cliente(conn_origem, conn_dest):
    cur_o = conn_origem.cursor()
    cur_o.execute("SELECT tb010_cpf, tb010_nome FROM tb010_clientes")
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    mapa = {}
    for cpf, nome in linhas:
        cur_d.execute("INSERT INTO cliente (nome) VALUES (%s)", (nome,))
        mapa[cpf] = cur_d.lastrowid
    conn_dest.commit()
    return mapa

def carregar_dim_loja(conn_origem, conn_dest):
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT l.tb004_cod_loja, c.tb002_nome_cidade, e.tb001_sigla_uf
        FROM tb004_lojas l
        JOIN tb003_enderecos e ON l.tb003_cod_endereco = e.tb003_cod_endereco
        JOIN tb002_cidades c ON e.tb002_cod_cidade = c.tb002_cod_cidade
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    mapa = {}
    for cod_loja, cidade, uf in linhas:
        cur_d.execute("INSERT INTO loja (cidade, uf) VALUES (%s, %s)", (cidade, uf))
        mapa[cod_loja] = cur_d.lastrowid
    conn_dest.commit()
    return mapa

def carregar_dim_canal(conn_dest):
    # cadastra fixamente os canais de loja fisica e virtual

```

```

cur = conn_dest.cursor()
mapa = {}
for nome in ["Loja Física", "Loja Virtual"]:
    cur.execute("INSERT INTO canal (nome_canal) VALUES (%s)", (nome,))
    mapa[nome] = cur.lastrowid
conn_dest.commit()
return mapa

def carregar_dim_mes(conn_dest):
    # carrega fixamente o nome dos meses
    nomes = ["Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho",
             "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"]
    cur = conn_dest.cursor()
    mapa = {}
    for numero, nome in enumerate(nomes, start=1):
        cur.execute("INSERT INTO mes (nome_mes) VALUES (%s)", (nome,))
        mapa[numero] = cur.lastrowid
    conn_dest.commit()
    return mapa

def carregar_dim_ano(conn_origem, conn_dest):
    cur_o = conn_origem.cursor()
    cur_o.execute("SELECT DISTINCT YEAR(tb010_012_data) FROM tb010_012_vendas") # so tem 2025
    anos = [linha[0] for linha in cur_o.fetchall()]

    cur_d = conn_dest.cursor()
    mapa = {}
    for ano in anos:
        cur_d.execute("INSERT INTO ano (pk_ano) VALUES (%s)", (ano,))
        mapa[ano] = ano # aqui a pk é o próprio ano
    conn_dest.commit()
    return mapa

# -----
# 3. carregando os dados na tabela fato
# 3.1 agrupa e soma com base no que a pergunta quer
# 3.2 insere com base nos ids mapeados nos mapas criados na etapa 2
# 3.3 grava tudo isso na fato_venda
# -----

def carregar_fato_tipo_categoria(conn_origem, conn_dest, mapa_tipo, mapa_categoria):
    """Pergunta 1: quantidade de vendas por tipo e categoria."""
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT sub.tipo, p.tb013_cod_categoria,
               SUM(v.tb010_012_quantidade) AS qtd,
               SUM(v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
        FROM tb010_012_vendas v
        JOIN tb012_produtos p ON v.tb012_cod_produto = p.tb012_cod_produto
        JOIN (
            SELECT DISTINCT tb012_cod_produto AS cod, 'Alimento' AS tipo FROM tb014_prd_alimentos
            UNION ALL
            SELECT DISTINCT tb012_cod_produto, 'Eletro' FROM tb015_prd_eletros
            UNION ALL
            SELECT DISTINCT tb012_cod_produto, 'Vestuario' FROM tb016_prd_vestuarios
        ) sub ON p.tb012_cod_produto = sub.cod
        GROUP BY sub.tipo, p.tb013_cod_categoria
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    for tipo, cod_categoria, qtd, valor in linhas:
        cur_d.execute("""
            INSERT INTO fato_venda
            (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
             fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
            VALUES (%s, %s, NULL, NULL, NULL, NULL, NULL, NULL, %s, %s)
            """, (mapa_tipo[tipo], mapa_categoria[cod_categoria], qtd, valor))
    conn_dest.commit()
    print(f"[tipo+categoria] {len(linhas)} linhas inseridas")

def carregar_fato_funcionario_mes_ano(conn_origem, conn_dest, mapa_funcionario, mapa_mes, mapa_ano):
    """Pergunta 2: valor das vendas por funcionário, hierárquico por tempo."""
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT v.tb005_matricula, MONTH(v.tb010_012_data), YEAR(v.tb010_012_data),
               SUM(v.tb010_012_quantidade) AS qtd,
    """)

```

```

        SUM(v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
    FROM tb010_012_vendas v
    GROUP BY v.tb005_matricula, MONTH(v.tb010_012_data), YEAR(v.tb010_012_data)
    """
)
linhas = cur_o.fetchall()

cur_d = conn_dest.cursor()
for matricula, mes, ano, qtd, valor in linhas:
    cur_d.execute("""
        INSERT INTO fato_venda
            (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
             fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
        VALUES (NULL, NULL, %s, NULL, NULL, NULL, NULL, NULL, %s, %s)
    """, (mapa_funcionario[matricula], mapa_mes[mes], mapa_ano[ano], qtd, valor))
conn_dest.commit()
print(f"[funcionario+mes+ano] {len(linhas)} linhas inseridas")

def carregar_fato_funcionario(conn_origem, conn_dest, mapa_funcionario):
    """Pergunta 3: Volume total por funcionário"""
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT v.tb005_matricula,
            SUM(v.tb010_012_quantidade) AS qtd,
            SUM(v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
        FROM tb010_012_vendas v
        GROUP BY v.tb005_matricula
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    for matricula, qtd, valor in linhas:
        cur_d.execute("""
            INSERT INTO fato_venda
                (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
                 fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
            VALUES (NULL, NULL, %s, NULL, NULL, NULL, NULL, NULL, %s, %s)
        """, (mapa_funcionario[matricula], qtd, valor))
    conn_dest.commit()
    print(f"[funcionario] {len(linhas)} linhas inseridas")

def carregar_fato_funcionario_loja(conn_origem, conn_dest, mapa_funcionario, mapa_loja):
    """Pergunta 4: Atendimentos por funcionário e localidade."""
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT v.tb005_matricula, f.tb004_cod_loja,
            SUM(v.tb010_012_quantidade) AS qtd,
            SUM(v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
        FROM tb010_012_vendas v
        JOIN tb005_funcionarios f ON v.tb005_matricula = f.tb005_matricula
        GROUP BY v.tb005_matricula, f.tb004_cod_loja
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    for matricula, cod_loja, qtd, valor in linhas:
        cur_d.execute("""
            INSERT INTO fato_venda
                (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
                 fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
            VALUES (NULL, NULL, %s, NULL, %s, NULL, NULL, NULL, %s, %s)
        """, (mapa_funcionario[matricula], mapa_loja[cod_loja], qtd, valor))
    conn_dest.commit()
    print(f"[funcionario+loja] {len(linhas)} linhas inseridas")

def carregar_fato_cliente_mes_ano(conn_origem, conn_dest, mapa_cliente, mapa_mes, mapa_ano):
    """
    Pergunta 5: Valor das ultimas vendas realizadas por cliente.
    Agrega as vendas por Cliente, Mes e Ano.
    """
    cur_o = conn_origem.cursor()
    cur_o.execute("""
        SELECT v.tb010_cpf, MONTH(v.tb010_012_data), YEAR(v.tb010_012_data),
            SUM(v.tb010_012_quantidade) AS qtd,
            SUM(v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
        FROM tb010_012_vendas v
        GROUP BY v.tb010_cpf, MONTH(v.tb010_012_data), YEAR(v.tb010_012_data)
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    for cpf, mes, ano, qtd, valor in linhas:

```

```

        cur_d.execute("""
            INSERT INTO fato_venda
                (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
                 fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
            VALUES (NULL, NULL, NULL, %s, NULL, NULL, %s, %s, %s, %s)
            """, (mapa_cliente[cpf], mapa_mes[mes], mapa_ano[ano], qtd, valor))
        conn_dest.commit()
        print(f"[cliente+mes+ano] {len(linhas)} linhas inseridas")

def carregar_fato_cliente_canal_mes_ano(conn_origem, conn_dest, mapa_cliente, mapa_canal, mapa_mes,
mapa_ano):
    """
    Pergunta 6: Clientes que mais compraram na loja virtual com valor acumulado por período.
    Agrega as vendas por Cliente, Mês e Ano.
    """
    cur_o = conn_origem.cursor()

    cur_o.execute("""
        SELECT
            sub.tb010_cpf,
            sub.canal,
            sub.mes,
            sub.ano,
            SUM(sub.qtd) AS qtd,
            SUM(sub.valor) AS valor
        FROM (
            SELECT
                v.tb010_cpf,
                CASE
                    WHEN l.tb010_cpf IS NOT NULL THEN 'Loja Virtual'
                    ELSE 'Loja Física'
                END AS canal,
                MONTH(v.tb010_012_data) AS mes,
                YEAR(v.tb010_012_data) AS ano,
                v.tb010_012_quantidade AS qtd,
                (v.tb010_012_quantidade * v.tb010_012_valor_unitario) AS valor
            FROM tb010_012_vendas v
            LEFT JOIN (SELECT DISTINCT tb010_cpf FROM tb011_logins) l
                ON v.tb010_cpf = l.tb010_cpf
        ) sub
        GROUP BY sub.tb010_cpf, sub.canal, sub.mes, sub.ano
    """)
    linhas = cur_o.fetchall()

    cur_d = conn_dest.cursor()
    for cpf, canal, mes, ano, qtd, valor in linhas:
        cur_d.execute("""
            INSERT INTO fato_venda
                (fk_tipo, fk_categoria, fk_funcionario, fk_cliente,
                 fk_loja, fk_canal, fk_mes, fk_ano, quantidade, valor)
            VALUES (NULL, NULL, NULL, %s, NULL, %s, %s, %s, %s, %s)
            """, (mapa_cliente[cpf], mapa_canal[canal], mapa_mes[mes], mapa_ano[ano], qtd, valor))
        conn_dest.commit()
        print(f"[cliente+canal+mes+ano] {len(linhas)} linhas inseridas")

# -----
# 4. orquestração
# 4.1 conecta com o banco
# 4.2 faz a carga nas dimensoes
# 4.3 faz a carga na fato com mapas e tal
# 4.4 finaliza e fecha as conexoes co m o banco
# -----

def main():
    conn_origem = conectar_origem()
    conn_destino = conectar_destino()

    # try except pq pode ter erros e tal, ai executa td de uma vez
    try:
        mapa_tipo = carregar_dim_tipo(conn_destino)
        mapa_categoria = carregar_dim_categoria(conn_origem, conn_destino)
        mapa_funcionario = carregar_dim_funcionario(conn_origem, conn_destino)
        mapa_cliente = carregar_dim_cliente(conn_origem, conn_destino)
        mapa_loja = carregar_dim_loja(conn_origem, conn_destino)
        mapa_canal = carregar_dim_canal(conn_destino)
        mapa_mes = carregar_dim_mes(conn_destino)
        mapa_ano = carregar_dim_ano(conn_origem, conn_destino)

        carregar_fato_tipo_categoria(conn_origem, conn_destino, mapa_tipo, mapa_categoria)
        carregar_fato_funcionario_mes_ano(conn_origem, conn_destino, mapa_funcionario, mapa_mes,
mapa_ano)

```

```

        carregar_fato_funcionario(conn_origem, conn_destino, mapa_funcionario)
        carregar_fato_funcionario_loja(conn_origem, conn_destino, mapa_funcionario, mapa_loja)
        carregar_fato_cliente_mes_ano(conn_origem, conn_destino, mapa_cliente, mapa_mes, mapa_ano)
        carregar_fato_cliente_canal_mes_ano(conn_origem, conn_destino, mapa_cliente, mapa_canal,
mapa_mes, mapa_ano)

        print("Carga concluída com sucesso.")
    except Exception as e:
        conn_destino.rollback()
        print(f"Erro na carga, rollback aplicado: {e}")
    finally:
        conn_origem.close()
        conn_destino.close()

if __name__ == "__main__":
    main()

```